Informe de arbitraje

Revista Faraute, Ciencias y Tecnología — Universidad de Carabobo

Manuscrito: «Análisis multicapa de operadores de PostgreSQL y almacenamiento CSI en Kubernetes: un marco de análisis y un estudio empírico de CloudNativePG bajo fallos inyectados».

Fecha del informe: 4 de octubre de 2026

Dictamen: REVISIÓN MAYOR

Estimados autores:

Agradezco la oportunidad de evaluar este manuscrito. El trabajo aborda un problema de interés práctico: el comportamiento de PostgreSQL gestionado por operadores en Kubernetes ante fallos. Lo hace con un diseño experimental cuidadoso en varios aspectos: verificador de transacciones propio, aislamiento del dominio de fallo, estadística no paramétrica adecuada a muestras pequeñas, declaración franca de las restricciones del entorno y depósito público del material.

Sin embargo, el hallazgo que el manuscrito presenta como central —que la indisponibilidad sostenida del primario no provoca promoción y que la variable que gobierna el *failover* es la «visibilidad» del fallo— admite una explicación alternativa que el diseño no descarta. Esa explicación es un artefacto del mecanismo de inyección. Además, el marco formal S = (O, K, M, D) no aporta contenido verificable, la capa de almacenamiento CSI no llega a ejercitarse en el experimento y varias afirmaciones técnicas sobre los operadores son inexactas. Por estas razones recomiendo una revisión mayor.

Las observaciones van numeradas (O, T y E) para que puedan responder a cada una en una carta de respuesta punto por punto. La sección final distingue los cambios obligatorios de los recomendados.

1. Valoración general

El artículo propone una taxonomía de operadores de PostgreSQL y de almacenamiento CSI, un marco descriptivo S = (O, K, M, D) con dimensiones e invariantes, y un estudio piloto sobre CloudNativePG (CNPG) 1.28.0 en un clúster productivo. Se inyectan tres fallos: eliminación del pod primario (F1, *pod-kill*), indisponibilidad del primario durante 10 minutos (F2, *pod-failure*) y partición de red mediante *NetworkPolicy* (F3). Un cuarto escenario de latencia de E/S (F4) resultó no ejecutable. Los resultados son: F1 promueve una réplica con RTO mediano de 7,91 s; F2 no promueve y el pod se recrea en sitio en 36,75 s; F3 no promueve y la indisponibilidad iguala la duración de la partición; el RPO es nulo en todos los casos. A partir de ello se propone un predicado de visibilidad V(fallo, K).

Fortalezas

- El verificador de transacciones (*tx-verifier*) con identificadores monótonos, reintento del mismo identificador y tratamiento explícito del «*commit* fantasma» es un diseño sencillo y correcto para medir pérdida de transacciones confirmadas.
- La medición del RTO con un único reloj del cliente evita los problemas de sincronización entre nodos.
- El aislamiento del dominio de fallo (cuotas, RBAC, doble filtro de selección, *dry-run*) está bien pensado para un entorno productivo.
- La estadística es apropiada para n = 10: medianas, IQR, intervalos de mediana de distribución libre, Mann–Whitney exacto, tamaño de efecto y cotas de Clopper–Pearson. He verificado las cifras principales y son correctas.
- Las limitaciones (colocalización en un nodo, operador único, F4 no ejecutable) se declaran con franqueza, y el resumen advierte que se sostiene el contraste, no las magnitudes absolutas.
- Se depositó en Zenodo el paquete de ejecución y se declaró el uso de IA generativa en la redacción.

Debilidades principales (en orden de gravedad)

1. El resultado de F2 es probablemente un artefacto de la inyección. Chaos Mesh implementa *pod-failure* sustituyendo la imagen del contenedor por una imagen `pause`, y CNPG trata un cambio de imagen como motivo de actualización y reinicia el primario en sitio. Esto explicaría la «recreación in situ» en 36,75 s dentro de una inyección de 10 minutos. Además, la documentación de CNPG establece que un fallo sostenido del contenedor `postgres` del primario sí dispara el *failover* (O9).
2. El predicado V(fallo, K) se construye *a posteriori* a partir de los tres escenarios y predice que no habrá promoción en un fallo de nodo, en contra de la documentación del operador. No ha sido sometido a ninguna prueba discriminante (O10).
3. La capa CSI no se ejercita: el pod nunca cambia de nodo, no hay *detach/attach* y el almacenamiento no varía. El componente M es constante en todo el experimento, de modo que el estudio no es «multicapa» en su parte empírica (O3, T4).
4. F3 no aísla el primario del plano de control. Un estudio independiente con aislamiento completo reprodujo *split-brain* y pérdida de escrituras en CNPG, lo que contradice la generalización de que CNPG «evita el *split-brain*» (T3).
5. El marco S = (O, K, M, D) y las Ecs. (2)–(5) son funciones no especificadas; no generan predicciones ni se contrastan (O5).
6. Varias afirmaciones técnicas son inexactas: el DCS de Patroni en Kubernetes, el valor de `node-monitor-grace-period`, el papel del *kubelet* en F1 y el de la reasociación CSI en el *failover* de CNPG (T1, T2, T4).

2. Observaciones sustantivas por sección

Título y resumen

O1. El título promete más de lo que el estudio entrega. El análisis empírico cubre un operador, un tipo de almacenamiento sin variar y un solo nodo. La comparación entre operadores y entre categorías de almacenamiento es documental. Sugiero un título centrado en lo medido, por ejemplo: «Comportamiento de *failover* de CloudNativePG ante fallos inyectados: estudio piloto en un clúster Kubernetes productivo». El resumen habla de «indisponibilidad sostenida» en F2, pero la indisponibilidad observada duró 36,75 s, no los 10 minutos de la inyección (O9).

Introducción y trabajos relacionados

O2. La «refutación del comportamiento documentado» se enuncia de forma contradictoria. La introducción presenta F2 como «una refutación, no una confirmación» del comportamiento documentado. La sección 6.3, en cambio, dice que la expectativa de promoción derivaba de tratar *pod-failure* como aproximación de un fallo de nodo y que «un lector familiarizado con *pod-failure* podría anticiparlo». Una de las dos lecturas debe prevalecer. Si se mantiene la refutación, citen el pasaje concreto de la documentación de CNPG que se refuta. La documentación de *failover* incluye entre los disparadores un fallo sostenido del contenedor `postgres` del primario, con `failoverDelay` igual a 0 s por defecto (CloudNativePG, *Failover*).

O3. La brecha declarada omite literatura directamente pertinente.

- Pruebas de fiabilidad de operadores de Kubernetes: Sieve (Sun et al., OSDI 2022) perturba el estado observado por los controladores para encontrar errores de fiabilidad. Acto (Gu et al., SOSP 2023) evaluó once operadores de sistemas con estado y encontró 56 errores nuevos. Ninguno incluye operadores de PostgreSQL, lo que permite delimitar mejor el aporte, pero ambos deben citarse.
- Pruebas de CNPG: la entrada de Drees (2026) no solo describe la eliminación repetida del primario con LitmusChaos. También integra pruebas de consistencia tipo Jepsen y métricas de tiempo de *failover* en el flujo automatizado. La afirmación de que ese trabajo no cuantifica el costo debe matizarse.
- *Split-brain* en CNPG: Sivko (2026) aisló el primario del API server, del operador y de las réplicas con Chaos Mesh. Con la configuración por defecto, el primario aislado siguió aceptando escrituras 99 s después de promoverse la réplica, con pérdida de escrituras confirmadas. Aunque es posterior a la redacción, debe incorporarse en la revisión porque condiciona la lectura de F3 (T3).
- Para el compromiso latencia–consistencia en ausencia de partición, PACELC (Abadi, 2012) es más adecuado que CAP.

O4. Hay citas que no sustentan lo que se afirma o son tangenciales.

- Stonebraker & Kemnitz (1991) describen POSTGRES, cuyo gestor de almacenamiento original era de no sobrescritura y no usaba WAL; el WAL se incorporó a PostgreSQL mucho después. Para el invariante de durabilidad basado en WAL, la referencia canónica es ARIES (Mohan et al., 1992).
- La replicación en cadena (Van Renesse & Schneider, 2004), CockroachDB (Taft et al., 2020), la consistencia eventual (Burckhardt, 2014) y la revisión SQL/NoSQL de Khan et al. (2023) no intervienen en el análisis. PostgreSQL usa replicación primario–respaldo por *streaming*. Reduzcan esas citas o expliquen su función.

Modelo del sistema y marco de evaluación

O5. El marco formal no tiene contenido verificable. Las Ecs. (2)–(5) expresan cada dimensión como f(·) de unas variables, y el propio texto aclara que esas funciones son «organizativas» y no relaciones definidas. Una ecuación cuya función no se especifica no aporta información. La Ec. (5), I(S) = f(O × K × M × D), usa un producto cartesiano de componentes sin significado operacional. Hay dos opciones. La primera es retirar las ecuaciones y presentar el modelo como un esquema conceptual (por ejemplo, una tabla de capas y responsabilidades). La segunda es darles contenido operacional, que el artículo ya tiene a mano:

- RPO = $|\mathrm{Tx}_{\text{confirmadas}} \setminus \mathrm{Tx}_{\text{visibles}}|$, el número de transacciones confirmadas y no visibles tras el fallo;
- RTO = $t_{\text{primer } \textit{commit} \text{ tras el fallo}} - t_{\text{último } \textit{commit} \text{ antes del fallo}}$;
- disponibilidad como fracción del tiempo de prueba con escrituras aceptadas.

O6. La dimensión de rendimiento P(S) se define pero no se mide. pgbench se ejecutó en todas las rondas y produce TPS y latencias por transacción. Reportarlos, al menos en régimen estable y durante la recuperación, daría contenido a P(S). En caso contrario, la dimensión debe eliminarse.

O7. La taxonomía es insuficiente como contribución. Clasificar tres operadores y tres categorías de almacenamiento sin criterios de clasificación explícitos no constituye una taxonomía. Además, la clasificación es inconsistente: Crunchy se describe en el texto como «intermedio», pero la Tabla 1 indica que, como Zalando, se apoya en Patroni. Y las categorías de almacenamiento se solapan: Ceph RBD es a la vez distribuido y en red, igual que una SAN Fibre Channel es almacenamiento en red. Propongo dimensiones explícitas (mecanismo de elección de líder, fuente de verdad del estado, modelo de almacenamiento compartido frente a *shared-nothing*, mecanismo de cercado) y ubicar en ellas cada operador.

O8. La sección 4 es especulativa y debería condensarse. Afirmaciones como «cabe esperar mayor coherencia» o «un comportamiento intermedio» se derivan de documentación y no se miden. La Tabla 1 resume documentación útil, pero no requiere

tres subsecciones de discusión. Sugiero reducir las secciones 3 y 4 a lo indispensable y dar más espacio al experimento.

Metodología experimental

O9. F2 debe reinterpretarse o repetirse con otro mecanismo de inyección. Esta es la observación más importante.

- Chaos Mesh implementa *pod-failure* modificando el campo `image` de los contenedores del pod para reemplazarlo por una imagen `pause`, y guarda la imagen original en anotaciones para restaurarla al final (Chaos Mesh, principios de implementación).
- CNPG inicia una actualización cuando la imagen del pod no coincide con la deseada. Por defecto, con `primaryUpdateStrategy: unsupervised` y `primaryUpdateMethod: restart`, reinicia en sitio el pod del primario (CloudNativePG, *Rolling updates*).
- Esto explica de forma natural lo observado: una inyección programada para 10 minutos que termina en 36,75 s con el pod recreado con el mismo nombre y el mismo PVC. El operador no «prefirió esperar», sino que deshizo la inyección al reconciliar una deriva de la especificación.

En consecuencia, F2 no mide una «indisponibilidad sostenida», no aproxima un fallo de nodo y no refuta el comportamiento documentado. Mide la reacción del operador a un cambio de imagen. Solicito:

1. los registros del operador durante F2, que muestren la causa registrada del reinicio;
2. el manifiesto completo del `Cluster`, con `failoverDelay`, `primaryUpdateStrategy`, `primaryUpdateMethod`, sondas y antiafinidad;
3. repetir F2 con un mecanismo que no altere la especificación del pod, por ejemplo terminación repetida del proceso `postgres` (*container-kill* en bucle o señal `SIGKILL` periódica) o bloqueo del puerto de las sondas.

Si con ese mecanismo CNPG promueve, el hallazgo central desaparece y el aporte pasa a ser la advertencia metodológica, que también es valiosa: *pod-failure* no es un modelo válido de fallo sostenido para operadores que reconcilian la imagen del pod. Aclaren además qué ocurrió con la imagen `pause` en un entorno *air-gapped*: si no estaba importada, el contenedor quedaría en `ImagePullBackOff`, lo que podría explicar el arranque en frío de 80,5 s de la primera inyección.

O10. El predicado V(fallo, K) no está contrastado y predice algo implausible. Sus tres valores (eliminado, *NotReady*-recreado, *Ready*) coinciden uno a uno con los tres escenarios observados, de modo que se ajusta a los datos por construcción. Además, si la promoción solo ocurre cuando el pod es eliminado, el predicado implica que CNPG no promoverá ante un fallo de nodo, en el que el pod persiste como objeto.

La documentación de CNPG indica lo contrario: el controlador del ciclo de vida del nodo marca el pod como no listo entre 40 y 55 s después de que el nodo deja de responder, y eso dispara el *failover*. Para sostener V, formulen predicciones *a priori* para escenarios que lo discriminen —fallo sostenido del contenedor sin cambio de imagen, parada del *kubelet*, apagado del nodo— y contrástenlas. Mientras tanto, preséntenlo como hipótesis.

O11. El RTO incluye el comportamiento del cliente, que no se describe. El RTO se mide como el hueco entre *commits* del verificador, de modo que incluye la detección por el operador, la promoción, la actualización de los *Endpoints* del servicio `-rw` y la reconexión del cliente. Indiquen el controlador de base de datos, los tiempos de espera de conexión y de consulta, el *keepalive* de TCP, la política de reintentos y si hubo *pooler*. Recomiendo descomponer el RTO de F1 en una línea de tiempo con los eventos de Kubernetes y del operador: inyección, evento «*Current primary isn't healthy*», promoción, cambio de *Endpoints* y primer *commit* aceptado.

O12. El RPO nulo tiene poco poder de detección en este diseño. Con las tres instancias en el mismo nodo, carga ligera (pgbench `-s 10`, 4 clientes) y un único escritor secuencial en el verificador, la ventana de pérdida posible es mínima y el retardo de replicación será casi nulo. Los propios autores lo atribuyen a un *lag* ≈ 0 no medido. Además, solo se verifican las transacciones del verificador, no las de pgbench. Recomiendo:

- registrar en cada inyección `pg_current_wal_lsn()` en el primario y `pg_last_wal_replay_lsn()` en las réplicas, para conocer el retardo en el instante del fallo;
- informar la tasa de escritura del verificador; la cifra de 613 253 identificadores no permite deducirla sin la duración total;
- en la Fase 2, aumentar la carga de escritura para que una pérdida con replicación asíncrona sea observable si ocurre.

O13. La ejecución en un clúster productivo requiere más información.

- Declaren la autorización formal de la organización para inyectar fallos en ese entorno.
- El operador CNPG era compartido con otros clústeres: indiquen cuántos, porque la carga de reconciliación del operador puede afectar la latencia de detección.
- El nodo y la SAN eran compartidos con cargas productivas: informen la carga de CPU y de E/S durante las rondas, o al menos su rango, porque es una fuente de varianza no controlada.
- La antiafinidad «solicitada» debía ser preferente y no obligatoria; de lo contrario, las réplicas no se habrían programado en un solo nodo. Precísenlo.

O14. El material suplementario sostiene demasiado del argumento. Los intervalos de confianza, el criterio de exclusión, la prueba de permutación, el detalle del aislamiento y el análisis de F4 se remiten al material suplementario. Con n = 10 por escenario, los 20 valores individuales de F1 y F2 caben en una tabla del texto principal. Confirmen también que el depósito de Zenodo citado está abierto y completo, y consideren publicar los registros crudos del verificador, debidamente anonimizados, en lugar de facilitarlos solo a petición: el RPO nulo descansa precisamente en ellos.

Resultados, discusión y conclusiones

O15. «Comportamiento CP» describe un caso en que no hubo decisión. En F3, las sondas del *kubelet* siguieron funcionando y el pod permaneció *Ready*: el operador no detectó ningún fallo, de modo que no «eligió» consistencia sobre disponibilidad. En sentido estricto, un primario con replicación asíncrona que promueve sin esperar confirmación de las réplicas (F1) acepta la posibilidad de perder transacciones confirmadas. Eso no es un comportamiento CP. Recomiendo describir F3 como «fallo no detectado por el plano de control» y reservar la terminología CAP para casos en que el sistema detecta la partición y decide.

O16. F3 no tiene RTO en sentido propio. No hubo acción de recuperación. La cifra (60 → 60,75 s) es la duración de la indisponibilidad, no un tiempo de recuperación. Usen esa denominación en la Tabla 2.

O17. La conclusión sobre el aporte debe ajustarse. Si se confirma la observación O9, el hallazgo de mayor «fuerza probatoria» desaparece. Aun si no se confirma, el predicado de visibilidad es una hipótesis (O10), no la «contribución conceptual de mayor alcance». Las afirmaciones de que el marco es «una herramienta reutilizable» y de que el trabajo «acerca la teoría de sistemas distribuidos a la práctica» exceden lo demostrado. La conclusión debería centrarse en lo medido: el RTO de F1 en colocalización, el comportamiento de F2 con su mecanismo real y la ausencia de detección en F3.

3. Aspectos técnicos de Kubernetes y de los operadores

T1. Patroni no usa etcd/Raft como DCS en los operadores citados. La Tabla 1 y la sección 4.3 contrastan Patroni, «que delega la fuente de verdad en un DCS por consenso (etcd/Raft)», con CNPG, que usa el API server. En Kubernetes, Patroni puede usar objetos de Kubernetes (*Endpoints* o *ConfigMaps*) como DCS sin desplegar etcd propio (documentación de Patroni). Esa es la configuración de Zalando Postgres Operator (Spilo) y de Crunchy PGO v5. Ambos enfoques dependen, por tanto, del API server, respaldado a su vez por el etcd del clúster. La diferencia real está en quién decide la promoción: Patroni en cada pod o el operador centralizado. El contraste debe corregirse.

T2. Hay imprecisiones en la Tabla 1.

- **Fallo del pod primario:** cuando el pod se elimina (F1), el *kubelet* no lo «reinicia». CNPG gestiona los pods directamente, sin *StatefulSet*, y es el operador quien crea el nuevo pod como réplica.
- **Fallo de nodo:** en versiones recientes de Kubernetes, incluida la v1.34 usada, el valor por defecto de `node-monitor-grace-period` es de 50 s y no de 40 s; la documentación de CNPG habla de 40 a 55 s para que el pod quede marcado como no listo.
- **Fallo de nodo con almacenamiento en red:** un volumen RWO de un nodo caído no se desasocia hasta que vence el tiempo de espera del controlador de *attach/detach* (6 min) o se aplica la marca `node.kubernetes.io/out-of-service` de apagado no ordenado. Este detalle es relevante para la Fase 2.

T3. F3 no prueba el caso difícil y su conclusión no se generaliza. La *NetworkPolicy* bloqueó el tráfico entre pods, pero el *failsafe* de Calico mantuvo el tráfico del *host*: las sondas siguieron respondiendo. CNPG 1.27 estabilizó y activó por defecto una comprobación de aislamiento del primario en la sonda de vida (`.spec.probes.liveness.isolationCheck`), que actúa cuando el primario pierde contacto con el API server y con las demás instancias (notas de la versión 1.27). Indiquen si la política bloqueaba el tráfico de salida hacia el API server y si esa comprobación llegó a activarse. Con aislamiento completo, Sivko (2026) observó una ventana de 99 s con dos primarios aceptando escrituras y pérdida de escrituras confirmadas, causada por la interacción entre `failoverDelay` (0 s) y `smartShutdownTimeout` (180 s). La afirmación de que CNPG «evita el *split-brain* apoyándose en el API server» debe restringirse al escenario efectivamente probado.

T4. La reasociación CSI no está en el camino crítico del *failover* de CNPG. El artículo afirma que F2 es una cota inferior del RTO ante fallo de nodo porque no captura la reasociación del volumen CSI, «dominante en un fallo de nodo genuino». CNPG sigue una arquitectura *shared-nothing*: cada instancia tiene su propio PVC y el *failover* promueve una réplica que ya dispone de su volumen. La reasociación del volumen del antiguo primario solo interviene en su reincorporación como réplica. En un fallo de nodo, el RTO de CNPG estaría dominado por la detección (40–55 s) más la promoción. Además, F2 sigue otro mecanismo (O9) y no puede actuar como cota de ese escenario. Corrijan el argumento en las secciones 5, 6 y 7.

T5. La configuración de almacenamiento debe justificarse frente a la recomendación del operador. La documentación de CNPG recomienda almacenamiento local, ya que la replicación la realiza PostgreSQL. El uso de una SAN Fibre Channel es legítimo, pero conviene discutir sus implicaciones: doble replicación, latencia y, con los pods colocalizados, que las tres instancias comparten el mismo dominio de fallo de almacenamiento.

T6. *Pod-kill* necesita su período de gracia. Indiquen el `gracePeriod` usado en PodChaos. Con un período de gracia mayor que cero, la terminación es ordenada y CNPG puede realizar un apagado limpio, lo que cambia el RTO y el riesgo de pérdida respecto de una terminación abrupta.

4. Aspectos estadísticos

E1. El tamaño de F2 no es coherente. Se indica que F2 se repitió diez veces y que la primera inyección (80,5 s) se excluyó. Si fueron diez en total, la mediana se calculó con nueve valores, y el p exacto de Mann–Whitney con U = 0 sería $\approx 2{,}2 \times 10^{-5}$ (10 frente a 9), no $\approx 1{,}1 \times 10^{-5}$ (10 frente a 10). Aclaren si hubo once inyecciones de F2.

E2. La exclusión del arranque en frío es una decisión *a posteriori*. El primer fallo en producción suele ocurrir precisamente en frío. Reporten la mediana de F2 con y sin esa ejecución y expliquen la causa del retraso (véase O9 sobre la imagen `pause`). La analogía con las dos ejecuciones exploratorias de F3 no es simétrica: estas tenían duraciones distintas por diseño, mientras que la primera de F2 es una repetición válida del mismo escenario.

E3. La precisión reportada excede la resolución del instrumento. Con una granularidad de ≈ 0,2 s, expresar el RTO con dos decimales (7,91 s; 36,75 s) es una precisión espuria. Redondeen a una décima o indiquen ± 0,2 s.

E4. La correlación orden–RTO no es «débil». Un $\rho = 0{,}62$ es una asociación moderada a fuerte; no es significativa con $n = 10$ por falta de potencia, no por su magnitud. Sugiere deriva entre repeticiones, que debe examinarse. Indiquen además si F1 y F2 se ejecutaron en bloques o intercalados, y el intervalo entre inyecciones: en bloques, la hora y la carga del clúster quedan confundidas con el escenario.

E5. La prueba de hipótesis aporta poco. F1 y F2 siguen mecanismos distintos (promoción frente a reinicio en sitio), y que sus RTO difieran es esperable. Los autores ya reconocen que el peso recae en el tamaño de efecto y el mecanismo. Basta con presentar los valores individuales y la diferencia de Hodges–Lehmann con su intervalo.

Comprobación de cifras

Recalculé las cifras que pueden verificarse con los datos del manuscrito.

Elemento	Reportado	Recalculado	Estado
Razón F2/F1	≈ 4,65×	36,75 / 7,91 = 4,65	Correcto
Mann–Whitney exacto, U = 0, 10 frente a 10	$p \approx 1{,}1 \times 10^{-5}$	$2 / C(20,10) = 1{,}08 \times 10^{-5}$	Correcto si $n_{F2} = 10$
Cota de Clopper–Pearson, 0/10	≤ 25,9 %	$1 - 0{,}05^{1/10} = 25{,}9\ \%$	Correcto
Cota de Clopper–Pearson, 0/12	≤ 22,1 %	$1 - 0{,}05^{1/12} = 22{,}1\ \%$	Correcto
Cobertura del IC de la mediana, n = 10	≈ 97,9 %	Estadísticos de orden 2 y 9: 97,85 %	Correcto
ρ crítico de Spearman, n = 10, α = 0,05	≈ 0,65	≈ 0,648	Correcto
Tamaño de F2 tras exclusión	n = 10	10 − 1 = 9 si hubo diez inyecciones	**Aclarar** (E1)
`node-monitor-grace-period`	≈ 40 s	50 s por defecto en versiones recientes	**Corregir** (T2)

5. Aspectos formales y editoriales

La redacción es clara, aunque densa en las secciones 3 y 4. Los ajustes formales son estos:

1. **Ecuaciones.** Retirar o dar contenido operacional a las Ecs. (2)–(5) (O5). La Ec. (6) es una definición de conjunto, no una ecuación.
2. **Figuras.** La Fig. 1 tiene texto ilegible al tamaño de impresión. La Fig. 2 advierte que sus hitos internos son esquemáticos: sustitúyanla por un gráfico de puntos con las diez ejecuciones de F1 y de F2. La Fig. 3 repite el texto y puede eliminarse o integrarse en una tabla.
3. **Tabla 2.** Renombrar la columna «RTO / indisp.» para distinguir recuperación de indisponibilidad (O16) y añadir el número de ejecuciones por escenario.
4. **Terminología.** Unificar «conmutación por error» y *failover*. Usar cursiva de forma consistente en los términos en inglés (*pod*, *commit*, *split-brain*). En la expresión «IQR ≈ [7,62; 8,01]» se informan los cuartiles, no el rango intercuartílico: escriban «Q1–Q3».

5. **Referencias.** Corregir la cita del WAL (O4), incorporar las omitidas (O3) y ajustar el estilo de la literatura gris, cuyas URL aparecen separadas por espacios en la bibliografía.
6. **Autoría.** Incluir el ORCID de todos los autores y preferir correos institucionales.
7. **Ética y autorización.** Añadir una declaración sobre la autorización para experimentar en un entorno productivo y sobre las medidas de contención (O13).

6. Recomendaciones y veredicto

Veredicto: REVISIÓN MAYOR. El diseño de medición es sólido y el trabajo puede convertirse en un buen reporte empírico. Sin embargo, el hallazgo central debe someterse a una prueba que descarte el artefacto de inyección antes de que pueda aceptarse. Si se confirma el artefacto, el artículo debe reorientarse.

Cambios obligatorios

1. Aportar los registros del operador en F2 y el manifiesto completo del `Cluster`, y repetir F2 con un mecanismo que no modifique la imagen del pod. Reinterpretar el resultado según lo que se obtenga (O9).
2. Presentar el predicado V(fallo, K) como hipótesis y contrastarlo con predicciones *a priori*, o retirarlo como contribución principal (O10, O17).
3. Corregir las afirmaciones técnicas sobre Patroni y el DCS, el *kubelet* en F1, `node-monitor-grace-period` y el papel de la reasociación CSI en el *failover* de CNPG (T1, T2, T4).
4. Restringir las conclusiones de F3 al aislamiento efectivamente probado, informar si el API server era alcanzable y si actuó la comprobación de aislamiento, e incorporar Sivko (2026) (T3, O15).
5. Retirar o dar contenido operacional al marco formal (O5) y ajustar el título y la taxonomía al alcance real (O1, O7).
6. Incorporar la literatura omitida (Sieve, Acto, pruebas de CNPG) y corregir las citas inadecuadas (O3, O4).
7. Aclarar el número de inyecciones de F2 y reportar las medianas con y sin la ejecución excluida (E1, E2).
8. Documentar el cliente de medición y descomponer el RTO de F1 en una línea de tiempo (O11).
9. Declarar la autorización para experimentar en producción y las condiciones de carga compartida (O13).

Cambios recomendados

1. Registrar el LSN en cada inyección y aumentar la carga de escritura para dar poder de detección al RPO (O12).
2. Reportar el rendimiento de pgbench o eliminar la dimensión P(S) (O6).
3. Incluir los valores individuales de F1 y F2 en el texto principal y verificar el acceso al depósito de Zenodo (O14).
4. Ajustar la precisión del RTO a la resolución del instrumento y examinar la deriva entre repeticiones (E3, E4).
5. Condensar las secciones 3 y 4 (O8).
6. Indicar el período de gracia de *pod-kill* y discutir la elección de la SAN frente al almacenamiento local (T5, T6).
7. Atender los aspectos formales de la sección 5.

Con estos cambios, el artículo ofrecería una caracterización empírica honesta del *failover* de CloudNativePG y una advertencia metodológica útil sobre la validez de los inyectores de fallos frente a operadores que reconcilian el estado de sus pods. Solicito que la versión revisada se acompañe de una carta de respuesta que atienda cada observación por su número. Quedo a disposición para evaluar la versión corregida.

Árbitro evaluador